Prompt Diff Debugger:

A Detailed Proposal for Section-Level Regression Localization, Validity-Preserving Search, and Minimal Repair in Long Structured System Prompts

Abstract

Modern prompt-management stacks already support prompt versioning, diffing, evaluation, rollback, and CI/CD gating, but they still leave a critical operational question mostly unresolved: when a long system prompt regresses, which prompt change caused the failure, and what is the smallest safe fix? Existing tooling such as LangSmith and Promptfoo makes it easier to detect prompt regressions and manage prompt versions, while recent research such as AgentStepper and DoVer shows that debugging LLM systems benefits from intervention and validation rather than log inspection alone [1, 2, 3, 5, 6]. This proposal introduces *Prompt Diff Debugger*, a practical system for localizing failures in long structured prompts to specific changed sections, preserving prompt validity during search, reducing search cost under CI/CD constraints, and recommending minimal rollbacks or targeted repairs. The project is intentionally narrow and practice-oriented. It focuses on long prompts used in structured-output assistants, coding assistants, and policy-heavy enterprise assistants; assumes black-box access to the underlying LLM; and relies on strong monitors such as JSON schema validation, code compilation and unit tests, and policy compliance checks.

The proposal makes three design decisions explicit. First, prompt reduction must avoid *invalid prompt states*: mixed old/new prompt variants should be filtered by a lightweight semantic consistency checker before expensive evaluation, analogous to how test-case reducers avoid syntactically invalid intermediate programs [7, 8, 9]. Second, search must be budgeted: the system should use representative failing examples, hierarchical replay, and hard time/API budgets so that localization can fit realistically into CI/CD workflows [3]. Third, few-shot demonstrations must be treated as structured arrays rather than monolithic text, enabling item-level isolation of harmful examples. The document provides a precise problem statement, a formal system model, detailed implementation roadmap, concrete JSON data formats, specific models and training plans, evaluation criteria, and deployment guidance.

1 Introduction and Motivation

Prompt engineering has become part of production software engineering. Teams now commit prompts to version control, compare prompt variants, pin or roll back prompt versions, and evaluate prompt changes before deployment. LangSmith explicitly supports prompt versions, diffs, prompt commits, and rollback workflows, while Promptfoo supports CI/CD evaluation and regression gates for LLM applications [1, 2, 3]. OpenAI's guidance similarly emphasizes building evals to monitor prompt performance as prompts and model versions change [4]. These capabilities answer *whether* a prompt change caused a regression. In practice, teams often need a more operational answer:

> *Which specific prompt section caused the regression, and what is the smallest safe edit that restores the prior behavior?*

This problem appears in several realistic workflows:

- a structured-output assistant loses JSON validity after an output-format section is simplified,
- a coding assistant loses unit-test performance after two demonstrations are removed,
- a policy-heavy enterprise assistant stops refusing a disallowed request after a safety section is rewritten,
- a citation-requiring report assistant starts omitting citations after prompt sections are reordered.

In these settings, the prompt is not just a string. It is closer to a *prompt program*: an ordered control artifact containing role framing, objectives, formatting constraints, examples, policies, workflow instructions, and exceptions. A single pass/fail signal at the whole-prompt level is therefore often too coarse.

Recent debugging research around LLM systems further motivates a more intervention-oriented approach. AgentStepper introduces an interactive debugger for software development agents, supporting stepwise inspection and intervention on agent trajectories [5]. DoVer proposes intervention-driven auto-debugging for multi-agent systems and shows that validated interventions are more useful than untested attribution hypotheses [6]. These systems focus on agent trajectories, plans, and messages, not on prompt-version diffs. Meanwhile, prompt-management systems focus on prompt lifecycle management rather than prompt-section fault localization. The proposed project occupies this gap.

Core thesis. The key thesis of this proposal is the following:

> *Long system prompt regressions can often be localized efficiently by treating prompts as structured programs, diffing them at the section and demonstration-item level, filtering out invalid mixed states, and replaying only a small, representative set of failing examples under strong monitors.*

2 Problem Statement

2.1 Practical problem

Let P^{old} be a prompt version that passes the relevant regression suite and let P^{new} be a later version that fails one or more evaluation cases. The engineering goal is to determine:

(a) which changed section or subset of sections is most likely responsible for the regression,

(b) whether reverting or repairing only that subset is sufficient to recover the desired behavior,

(c) what minimal rollback or targeted patch restores the failing metrics while preserving the desired new changes.

2.2 Formal problem formulation

Assume a black-box LLM service

$$F(P, x, \omega) \to y,$$

where P is a prompt, x is a task input, ω are execution settings, and y is the resulting output.

Let $\mathcal{D} = \{(x_i, m_i)\}_{i=1}^{N}$ be an evaluation dataset, where m_i encodes the monitor bundle for example i, such as a JSON schema, a set of unit tests, a policy checker, or a scoring rubric. Define a monitor-based pass function

$$\text{Pass}(P, x_i, m_i) \in \{0, 1\}.$$

Let a structured segmentation function map a prompt into typed sections:

$$\mathrm{Seg}(P) = (s_1, \ldots, s_M).$$

For demonstration sections, we further allow nested item decomposition:

$$\mathrm{Items}(s_j) = (e_{j1}, \ldots, e_{jL_j}).$$

Let $\Delta(P^{\mathrm{old}}, P^{\mathrm{new}})$ denote the set of changed units, where a unit may be a section or, if the section family is `demonstration`, an individual example item.

Let $\mathcal{F}_{\mathrm{fail}} \subseteq \mathcal{D}$ be the failing examples under P^{new}. The system seeks a repair operator R acting on a subset of changed units such that the repaired prompt

$$P^{\mathrm{fix}} = R(P^{\mathrm{new}})$$

restores performance while keeping the edit small. A natural objective is:

$$R^* = \operatorname*{argmin}_R \ \lambda_1 \,\mathrm{EditCost}(R) + \lambda_2 \,\mathrm{CallCost}(R) + \lambda_3 \,\mathrm{TimeCost}(R)$$

subject to

$$\frac{1}{|\mathcal{F}_{\mathrm{fail}}|} \sum_{(x_i, m_i) \in \mathcal{F}_{\mathrm{fail}}} \mathrm{Pass}(P^{\mathrm{fix}}, x_i, m_i) \geq \tau,$$

where τ is a recovery threshold chosen by the application team.

2.3 Validity-preserving search requirement

Unlike classical text diffing, some mixed old/new prompt states are semantically nonsensical. For example:

- a format section may require keys that are no longer introduced by the task section,
- a workflow section may refer to steps that only exist in the new policy section,
- a demonstration item may be inconsistent with the role or output schema,
- two policy sections may issue contradictory instructions.

Therefore the search space must include a third test outcome:

$$T(P, x_i, m_i) \in \{\mathrm{Pass}, \mathrm{Fail}, \mathrm{Invalid}\}.$$

A candidate mixed-state prompt should be marked Invalid if it violates structural or semantic consistency rules before full evaluation. This is directly analogous to structured test-case reduction techniques, which avoid spending effort on syntactically invalid intermediate reductions [8, 9].

2.4 Project goal

The project goal is to build a system that, given two prompt versions and failing evaluations, returns:

- a ranked list of likely failure-inducing sections or demonstration items,
- a minimal failure-inducing diff when possible,
- a validity-preserving rollback or patch recommendation,
- supporting evidence from replayed tests and monitors,
- a machine-readable debugging report for CI/CD and developer review.

3 Scope, Intended Use, and Non-Goals

3.1 Intended use

The intended deployment setting is a team that:

- maintains long, versioned system prompts,
- runs offline evals or CI/CD gates on prompt changes,
- has strong monitors rather than purely subjective judgments,
- wants faster prompt regression diagnosis than manual trial-and-error.

The most suitable first domains are:

(i) **structured-output assistants** returning JSON or XML,

(ii) **coding/debugging assistants** evaluated with compilation and tests,

(iii) **policy-heavy enterprise assistants** with explicit refusal, routing, or formatting rules.

3.2 Non-goals

The first version explicitly does *not* aim to:

- solve general prompt optimization,
- explain every model failure independent of prompt edits,
- handle highly open-ended creative tasks with weak evaluation,
- guarantee global semantic equivalence between prompt versions,
- replace existing prompt-management stacks.

4 Related Work and Positioning

Prompt versioning and prompt evaluation. LangSmith supports prompt versions, commits, diffs, and rollbacks, while Promptfoo supports CI/CD testing and regression gates for prompt-based systems [1, 2, 3]. These systems are highly relevant operationally, but they mainly identify that a version regressed, not which prompt section caused the regression.

Prompt engineering with evals. OpenAI's prompt-engineering guidance explicitly recommends building evals to monitor prompt performance as prompts evolve and as deployed model versions change [4]. This provides the operational setting for the proposed system.

Agent debugging. AgentStepper and DoVer show that debugging LLM systems benefits from controllable interventions and verification, rather than post hoc explanation alone [5, 6]. Their target is the execution trace or trajectory; ours is the prompt-version diff.

Delta debugging and structured reduction. Delta debugging isolates minimal failure-inducing differences by repeated automated testing [7]. Hierarchical Delta Debugging (HDD) exploits tree structure to reduce search cost and improve validity of intermediate candidates [8]. Perses further shows that syntax-guided reduction can avoid invalid intermediate states during program reduction

[9]. These ideas directly motivate the proposal's validity-preserving search over section graphs and demonstration arrays.

Positioning claim. The novelty claim is intentionally narrow:

> *Prompt Diff Debugger is not a claim to invent prompt versioning, prompt evaluation, or LLM debugging in general. Its contribution is a practical, section-level regression-localization framework for long structured prompts, with explicit validity checks, bounded search, and item-level treatment of few-shot demonstrations.*

5 Prompt Intermediate Representation

A core design choice is to avoid diffing prompts as raw strings. Instead, the system compiles each prompt into a *Prompt Intermediate Representation* (PIR).

5.1 PIR schema

Each prompt is represented as an ordered tree or graph:

$$\mathrm{PIR}(P) = (V, E, \prec, \tau, \kappa),$$

where:

- V is the set of prompt units,
- E is a dependency relation,
- $\prec$ is the canonical order relation,
- $\tau(v)$ is the type or family of node v,
- $\kappa(v)$ stores node-level metadata such as required keys, referenced steps, example labels, and confidence.

5.2 Node families

The initial family taxonomy is intentionally small:

Family	Meaning
`role`	persona and high-level behavior framing
`objective`	task objective or mission statement
`format`	output schema, serialization, or style constraints
`demonstration`	few-shot examples, exemplars, or templates
`policy`	safety, refusal, or compliance instructions
`workflow`	step-by-step procedure, decision sequence, or routing logic
`exception`	edge-case or fallback rule
`unknown`	abstention class

5.3 Demonstration sections as arrays

The proposal adopts the user's suggestion explicitly: demonstration sections must not be treated as one undifferentiated text block. Instead, a demonstration node contains an array of example items:

$$\text{Items}(s^{\text{demo}}) = (e_1, \ldots, e_L),$$

where each item has fields such as input, output, label, rationale, and formatting pattern. This enables item-level diffs and hierarchical localization. Since demonstrations often dominate the token budget of a long prompt, treating them as structured arrays is critical for both usefulness and efficiency.

6 System Architecture

The full system contains seven stages.

6.1 Stage A: Parsing and segmentation

A hybrid parser converts raw prompt text into candidate sections.

Rule-based front end. The deterministic parser identifies headings, numbered lists, markdown bullets, XML-like tags, fenced code and JSON blocks, explicit labels such as "Role:" or "Output format:", and separators between examples.

Learned refinement. A lightweight boundary refinement model optionally merges or splits ambiguous blocks.

Output. The stage outputs section spans, candidate family labels, parser confidence, and nested example items for demonstration sections.

6.2 Stage B: Section alignment and diffing

Given old and new PIRs, the aligner computes correspondences among sections and demonstration items. The diff engine distinguishes:

- unchanged sections,
- inserted and deleted sections,
- modified sections,
- reordered sections,
- modified, inserted, deleted, or reordered demonstration items.

Alignment uses lexical overlap, normalized edit distance, embedding similarity, and family priors.

6.3 Stage C: Semantic consistency checking

This stage is the main improvement prompted by the user's feedback.

Purpose. Before any expensive LLM replay, the system rejects mixed old/new prompt states that are structurally or semantically invalid.

Validator design. The consistency checker is lightweight and layered:

(1) **Structural rules:** required section families, mandatory ordering constraints, demonstration array integrity, schema field references.

(2) **Dependency rules:** if a format section refers to keys or steps, those must exist upstream; if a workflow step is referenced, the defining section must be present.

(3) **Contradiction rules:** known contradictory instruction pairs, such as "always cite sources" versus "do not include citations," or incompatible JSON key sets.

(4) **Optional learned contradiction filter:** a small NLI-style classifier that flags likely semantic contradictions among pairs of changed sections.

Output. Each candidate prompt variant receives one of three states:

`valid`, `suspicious`, `invalid`.

Only valid or high-confidence suspicious states proceed to replay.

Why this matters. This step plays the role of a compile check in delta debugging. Without it, the system wastes API calls on nonsensical mixed states and may misattribute regressions to artifacts of invalid prompt assembly rather than to real causal changes.

6.4 Stage D: Budgeted localization search

This stage identifies the smallest set of changed units likely to explain the regression.

6.4.1 Representative failing set selection

To keep CI/CD latency practical, the debugger does *not* localize over the entire failing suite by default. Instead it selects at most K representative failing examples, with K small (for example $K = 5$). The representatives are selected by clustering failure traces or monitor signatures so that each cluster contributes one representative.

Formally, let $\mathcal{F}_{\text{fail}}$ be the failing set. A selector chooses

$$\mathcal{F}_{\text{rep}} \subseteq \mathcal{F}_{\text{fail}}, \qquad |\mathcal{F}_{\text{rep}}| \leq K.$$

A practical policy is to cluster examples using embeddings of the input query plus the monitor-failure vector, then pick cluster medoids.

6.4.2 Search strategy

The system uses a coarse-to-fine strategy:

(1) **Section-level search** over changed top-level sections.

(2) **Hierarchical refinement** inside large changed sections, especially demonstration arrays.

(3) **Patch verification** on the larger failing set after a candidate culprit is found.

The preferred algorithm is a budgeted variant of delta debugging:

- start from all changed units,

- test subsets and complements,
- prune invalid candidates using the consistency checker,
- recurse only on plausible failure-inducing subsets,
- stop when the call or time budget is exhausted.

6.4.3 Demonstration-aware hierarchical search

For demonstration sections, the debugger applies item-level reduction rather than raw text diffing. This is conceptually closer to Hierarchical Delta Debugging than flat ddmin because the search respects the structure of the example array [8]. The search can also test grouped deletion by semantic clusters of examples, such as all error-handling examples or all JSON-format exemplars.

6.4.4 Search-space heuristics

The proposal adopts explicit heuristics to limit cost:

- do not localize on more than K representative failures during the first pass,
- prioritize changed units whose family matches the failing monitor type,
- skip unchanged units and units with no dependency path to the failing output contract,
- run section-level search before item-level search,
- cache replay results by candidate prompt hash,
- enforce global budgets such as at most B_{calls} model calls or B_{time} wall-clock seconds.

A realistic CI/CD target is a first-pass diagnosis within a small time envelope, such as under five minutes for a moderate-sized failing change set. Promptfoo itself emphasizes CI/CD gating and early regression detection, making bounded cost a necessary engineering requirement rather than an optional optimization [3].

6.5 Stage E: Culprit ranking and minimal diff extraction

After replay, the system ranks changed units by causal plausibility. Each candidate receives an evidence score based on:

- how often reverting it flips failures to passes,
- how small the required subset is,
- whether its family semantically matches the monitor failure,
- whether the effect generalizes beyond the representative failing set,
- whether the candidate survives validity checks.

The top result is returned together with a minimal diff whenever the search reaches a locally minimal failure-inducing subset.

6.6 Stage F: Patch recommendation

Patch generation is deliberately conservative.

Primary patch actions.

- exact rollback of the culprit unit,
- partial rollback of a demonstration item or field,
- restoration of an explicit schema clause,
- restoration of a deleted example,
- movement of a section back to its prior order,
- restoration of a missing dependency anchor.

Optional generative repair. After the rule-based library is implemented, an LLM-based rewrite fallback may be added for cases where exact rollback is undesirable but a templated repair is possible.

6.7 Stage G: Validation and report generation

Every candidate fix is re-evaluated on:

(1) the representative failing set,

(2) the full failing set if the first pass succeeds,

(3) optionally a guardrail or sanity suite of previously passing examples.

The final report contains:

- culprit ranking,
- minimal diff if found,
- replay evidence,
- recovered metrics,
- applied budget,
- recommended patch,
- machine-readable trace.

7 Data Design and Formats

The project requires five core datasets or data stores.

7.1 Dataset A: Prompt version corpus

Each prompt version is stored as a JSON object. A representative schema is shown below.

Listing 1: Prompt version record

```
{
  "prompt_id": "support_bot_v1",
  "version_id": "2026-04-02.3",
  "domain": "structured_output_support",
```

```
  "created_at": "2026-04-02T14:12:00Z",
  "raw_text": "...full prompt text...",
  "sections": [
    {
      "section_id": "s1",
      "family": "role",
      "title": "Role",
      "text": "You are a customer support triage assistant...",
      "required": true,
      "depends_on": []
    },
    {
      "section_id": "s2",
      "family": "format",
      "title": "Output format",
      "text": "Return valid JSON with keys action, priority, reason, customer_reply.",
      "required": true,
      "depends_on": ["s1"]
    },
    {
      "section_id": "s3",
      "family": "demonstration",
      "title": "Examples",
      "items": [
        {
          "item_id": "ex1",
          "input": "Customer asks for refund after duplicate charge.",
          "output": "{\"action\":\"refund_review\", ...}",
          "tags": ["billing", "refund"]
        }
      ],
      "required": false,
      "depends_on": ["s1", "s2"]
    }
  ]
}
```


7.2 Dataset B: Evaluation dataset

Each evaluation case includes a task input and monitor definitions.

Listing 2: Evaluation case record


```
{
  "eval_id": "json_support_0042",
  "family": "structured_output_support",
  "input": "Customer says they were charged twice and wants a refund.",
  "monitors": {
    "json_schema": {
      "type": "object",
      "required": ["action", "priority", "reason", "customer_reply"]
    },
    "field_rules": {
      "priority": ["low", "medium", "high"]
```

```
    }
  },
  "metadata": {
    "difficulty": "medium",
    "topic": "billing"
  }
}
```

7.3 Dataset C: Regression scenario dataset

Each scenario links two prompt versions with observed failures.

Listing 3: Regression scenario record

```
{
  "scenario_id": "reg_0137",
  "old_version": "2026-03-28.1",
  "new_version": "2026-04-02.3",
  "failing_eval_ids": ["json_support_0042", "json_support_0110"],
  "passing_baseline": true,
  "failure_signature": {
    "json_schema_fail": 2,
    "missing_required_fields": 2
  },
  "root_cause_label": {
    "section_ids": ["s2"],
    "granularity": "section"
  }
}
```

7.4 Dataset D: Consistency-check dataset

If the optional contradiction classifier is trained, it needs pairwise examples of section pairs labeled as:

- `consistent`,
- `possibly contradictory`,
- `contradictory`.

These can be synthesized by mutating prompt sections (for example by inverting schema keys, adding conflicting policy statements, or swapping incompatible workflow descriptions) and then manually validating a smaller subset.

7.5 Dataset E: Debug trace dataset

The debugger should log all replay attempts, making later supervised learning possible.

Listing 4: Replay trace record

```
{
  "trace_id": "trace_4412",
  "scenario_id": "reg_0137",
```

```
  "candidate_units": ["s2"],
  "candidate_type": "revert",
  "validity_state": "valid",
  "eval_subset": ["json_support_0042"],
  "result": "pass",
  "latency_ms": 1840,
  "api_calls": 1
}
```

8 Model Choices

The project should avoid unnecessary model complexity. A strong MVP can be mostly rule-based with a few small learned components.

8.1 Component 1: Segmentation/refinement model

MVP. Rule-based segmentation only.

Optional learned model. A small boundary classifier over candidate blocks.

Model choices.

- encoder: MiniLM, bge-small, or e5-small embeddings,
- classifier: logistic regression, linear SVM, or small MLP.

Training requirement. Optional. If used, train on manually annotated prompt sections. A few thousand labeled blocks are likely sufficient because the taxonomy is small and formatting cues are strong.

8.2 Component 2: Section-family classifier

Purpose. Assign each section to one of the predefined families.

Model choices.

- embeddings from MiniLM, bge-small, or e5-small,
- classifier: logistic regression or gradient boosted trees over embeddings plus structural features.

Training requirement. Recommended but lightweight. Freeze the encoder and train only the classifier. Use class-balanced loss or reweighting because some families, especially `exception`, may be rare.

8.3 Component 3: Alignment scorer

Purpose. Align old and new sections and demonstration items.

Model choices. For the MVP, use a weighted similarity score combining lexical overlap, normalized edit distance, embedding cosine similarity, and family agreement. A learned pair scorer may be added later.

Training requirement. Not required for the MVP. Optional pairwise ranking can be trained from aligned section pairs harvested from historical prompt revisions.

8.4 Component 4: Consistency validator

MVP. No training required. Use rule-based checks:

- missing required sections,
- broken dependencies,
- missing referenced keys or steps,
- contradictory known template pairs,
- malformed demonstration arrays.

Optional learned contradiction model. A compact NLI-style classifier over pairs of sections can serve as a soft contradiction filter.

Training requirement. Optional. If trained, use synthetic section-pair mutations plus human verification. Fine-tuning a small encoder with a three-way label space (consistent / suspicious / contradictory) is likely sufficient.

8.5 Component 5: Culprit ranker

Purpose. Rank changed units after replay.

Features.

- family type,
- change type (insert, delete, edit, move),
- edit size,
- dependency centrality,
- whether reversion flips pass/fail,
- consistency state,
- failing monitor signature.

Model choices. Gradient boosted trees or a linear ranker are suitable.

Training requirement. Optional. Train on historical or synthetic regressions. The labels can be root-cause sections if available, or a weak label based on successful minimal reverts.

8.6 Component 6: Patch recommender

MVP. A rule-based patch library only.

Optional extension. An LLM-based patch writer that receives the culprit section, dependency context, and monitor failure, and proposes a rewrite. The candidate rewrite must still pass the validator and the monitor-based replay.

Training requirement. Not required. This component can use in-context prompting rather than fine-tuning.

9 Training Plan

9.1 What requires training

For a realistic first implementation, training is *optional* for the MVP. The only components that benefit clearly from training are:

(1) section-family classifier,

(2) boundary refinement model,

(3) optional contradiction detector,

(4) optional culprit ranker.

9.2 Supervision sources

Training data can come from four sources:

- manual annotation of prompt sections,
- synthetic prompt mutations that create regressions,
- historical prompt revisions paired with eval outcomes,
- replay traces generated by the debugger itself.

9.3 Recommended training workflow

9.3.1 Phase 1: Section annotation

Annotate 3,000–6,000 sections from one or two narrow domains. Split into train/dev/test sets by prompt rather than by section to avoid leakage.

9.3.2 Phase 2: Synthetic regression generation

Generate realistic prompt mutations:

- remove required JSON keys from the format section,
- delete demonstration items,
- reorder workflow steps,
- weaken safety wording,
- replace explicit format instructions with vague style instructions.

Run evals to identify which mutations produce measurable regressions.

9.3.3 Phase 3: Contradiction dataset creation

Construct section-pair examples by mixing old and new sections under known dependency templates. Label a subset manually to calibrate the rule engine and optional contradiction classifier.

9.3.4 Phase 4: Replay-log learning

After the MVP is running, accumulate replay traces and successful fixes. Use these to train the culprit ranker and improve patch prioritization.

9.4 Losses and optimization

Section classifier. Use standard cross-entropy with class weighting.

Contradiction classifier. Use three-way cross-entropy and optimize for high recall on the `contradictory` class; false negatives are more costly than false positives because they waste API calls.

Culprit ranker. Use pairwise ranking loss or binary classification over candidate units labeled as successful versus unsuccessful repair targets.

10 Implementation Roadmap

10.1 Phase 0: Domain narrowing

Choose one or two domains only. Recommended combinations are:

- structured-output support assistant,
- coding/debugging assistant.

This keeps monitor design strong and prompt structure regular.

10.2 Phase 1: MVP (rule-based core)

Deliverables.

- prompt parser and section extractor,
- demonstration item extractor,
- diff engine,
- rule-based consistency checker,
- replay harness,
- delta-debugging-style search with time and call budgets,
- machine-readable debug report.

No training required. This phase should work with deterministic logic plus monitor-based replay.

10.3 Phase 2: Lightweight learned components

Deliverables.

- section-family classifier,
- section alignment scorer,
- contradiction filter,
- culprit ranker.

Goal. Improve coverage, reduce false positives, and reduce search calls.

10.4 Phase 3: Repair support and productization

Deliverables.

- patch template library,
- optional LLM rewrite fallback,
- CI/CD integration,
- prompt-version dashboard,
- replay cache and audit trail.

Goal. Turn localization into a practical developer workflow rather than a research demo.

11 Evaluation Plan

11.1 Primary questions

The evaluation should answer five questions:

(Q1) Can the system localize the failure-inducing prompt unit accurately?

(Q2) Does validity filtering reduce wasted API calls without hiding true culprits?

(Q3) Do representative failing examples preserve localization quality while reducing cost?

(Q4) Does demonstration-array handling outperform flat-text demonstration diffing?

(Q5) Can the debugger produce repairs that actually restore performance?

11.2 Metrics

Localization quality.

- top-1 culprit accuracy,
- top-3 culprit recall,
- minimal diff size,
- intersection-over-union with labeled culprit sets.

Search efficiency.

- number of model calls,
- wall-clock time,
- number of invalid candidates pruned,
- cache hit rate.

Repair utility.

- fraction of regressions repaired,
- recovered hard-monitor pass rate,
- recovered soft metric score,
- patch size.

Operational usefulness.

- mean time to diagnosis,
- mean time to passing patch,
- developer acceptance of recommended fix.

11.3 Baselines

The proposal should be compared against:

(i) whole-prompt rollback recommendation only,

(ii) plain text diff without replay,

(iii) random changed-section ablation,

(iv) full-suite exhaustive replay without representative-failure selection,

(v) flat demonstration diff rather than item-level demonstration diff.

11.4 Ablation studies

At minimum, evaluate:

- with and without consistency validator,
- with $K = 1, 3, 5, 10$ representative failures,
- section-level only versus section + demonstration-item search,
- rule-based ranker versus learned culprit ranker.

12 Feasibility Assessment

Why the project is feasible. The project is feasible because:

- it can deliver useful value with mostly rule-based components,
- the first target domains have strong monitors,
- prompt structure is regular enough in narrow domains,
- training is optional for the MVP,
- existing eval pipelines already provide the trigger signal.

Main risks. The main risks are:

- poor segmentation on messy prompts,
- weak or incomplete consistency rules,
- insufficiently representative failing examples,
- few-shot examples whose effects are highly interaction-dependent,
- lack of real historical regressions for evaluation.

Why the proposed improvements matter. The three improvements requested by the user substantially strengthen feasibility:

(1) **semantic consistency filtering** prevents the search from wasting effort on nonsensical mixed states,

(2) **bounded representative search** makes the workflow compatible with CI/CD,

(3) **demonstration arrays** turn the most token-heavy sections into tractable structured units.

13 Expected Contributions

If successful, the project contributes:

1. a practical formulation of prompt regression localization as structured, validity-preserving reduction,
2. a prompt intermediate representation with typed sections and demonstration arrays,
3. a bounded, CI-friendly localization algorithm,
4. a repair-oriented debugging workflow for real prompt-management pipelines,
5. datasets and traces for future supervised learning of prompt fault localization.

14 Conclusion

This proposal develops a narrow but useful system for real-world LLM engineering. Existing tooling already supports prompt versioning, evaluation, and rollback; the missing piece is reliable localization of prompt regressions to concrete prompt sections. Prompt Diff Debugger addresses that

gap by treating long prompts as structured programs, performing typed section and demonstration-item diffs, pruning invalid mixed states before replay, localizing failures under explicit API and time budgets, and recommending minimal repairs validated by strong monitors. The project is intentionally scoped so that it can work in practice: training is optional for the MVP, the first domains are chosen for strong automated oracles, and the search strategy is explicitly designed for CI/CD deployment.

References

References

[1] LangChain. *Manage prompts - LangSmith documentation*. Accessed April 2026. https://docs.langchain.com/langsmith/manage-prompts

[2] LangChain. *Prompt engineering concepts - LangSmith documentation*. Accessed April 2026. https://docs.langchain.com/langsmith/prompt-engineering-concepts

[3] Promptfoo. *CI/CD Integration for LLM Evaluation and Security*. Accessed April 2026. https://www.promptfoo.dev/docs/integrations/ci-cd/

[4] OpenAI. *Prompt engineering guide - OpenAI API documentation*. Accessed April 2026. https://developers.openai.com/api/docs/guides/prompt-engineering/

[5] Robert Hutter and Michael Pradel. *AgentStepper: Interactive Debugging of Software Development Agents*. arXiv:2602.06593, 2026. https://arxiv.org/abs/2602.06593

[6] Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. *DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems*. arXiv:2512.06749, 2025. https://arxiv.org/abs/2512.06749

[7] Andreas Zeller and Ralf Hildebrandt. *Simplifying and Isolating Failure-Inducing Input*. IEEE Transactions on Software Engineering, 28(2):183–200, 2002. https://www.cs.purdue.edu/homes/xyzhang/fall07/Papers/delta-debugging.pdf

[8] Ghahraman Misherghi and Zhendong Su. *HDD: Hierarchical Delta Debugging*. In *Proceedings of the 28th International Conference on Software Engineering*, 2006. https://people.inf.ethz.ch/suz/publications/icse06-hdd.pdf

[9] Chengnian Sun, Yuanbo Li, Qirun Zhang, Tianxiao Gu, and Zhendong Su. *Perses: Syntax-Guided Program Reduction*. In *Proceedings of the 40th International Conference on Software Engineering*, 2018. https://people.inf.ethz.ch/suz/publications/perses.pdf